

TP Réseau Docker

Contexte Pro :

Une architecture micro-service est un type d'architecture d'application dans laquelle l'application est développée sous forme d'un ensemble de service.

DevOps : est un mouvement en ingénierie informatique et une pratique technique visant à l'unification du développement logiciel et de l'administration des infrastructures informatiques notamment l'administration systèmes.

1. Créez un réseau Docker nommé "monitoring_network" pour simuler l'environnement de production.
 - **Docker network create --driver bridge monitoring_network ==> cette commande permet de créer un réseau docker**
 - **Docker network ls ==> permet d'afficher les réseaux docker créés**

```
C:\Users\mamad>docker network ls
NETWORK ID      NAME                DRIVER  SCOPE
3ff13cd89a0d    bridge             bridge  local
7e9e99df1327    host               host    local
0897ebb8ae94    monitoring_network bridge  local
2f7564dd460f    none               null    local
```

2. Créez un volume Docker nommé "shared_data" pour le partage sécurisé de fichiers entre les services.

Volume docker représente la méthode recommandée par docker pour la persistance dans les conteneurs.

- **Docker volume create "shared_data" ==> permet de créer le volume**
- **Docker volume ls ==> permet d'afficher les volumes créées**

```
root@d6a6e837436f: /
C:\Users\mamad>docker volume ls
DRIVER  VOLUME NAME
local   shared_data
```

3. Créez deux conteneurs Docker en mode interactif, nommés "file-generator" et "file-monitor", connectés au réseau "monitoring_network" et montant le volume "shared_data".

```
C:\Users\mamad>docker run -it --name file-generator --network monitoring_network -v shared_data:/app/data ubuntu /bin/bash
root@d6a6e837436f:/# sudo apt update
```

On fait la même commande avec l'autre conteneur file-monitor. Cette commande permet de créer le conteneur file-generator.

--network ==> permet de connecter le conteneur au réseau docker
"monitoring_network"

-v shared_data:/app/data ==> cette option monte un volume nommé
shared_data dans le conteneur à l'emplacement /app/data

Ubuntu ==> c'est l'image

4. Installez les outils nécessaires (sudo, cron, nano) via apt-get install

On fait les commandes suivantes dans chaque conteneur 1 et 2 :

- apt update
- Apt-get install sudo, cron et nano

5. 5. Vérifiez que les deux conteneurs peuvent communiquer entre eux en utilisant la commande ping.

- Apt install -y iputils -ping ==> permet d'installer les paquets pour la commande ping
- Ping file-generator dans le conteneur fileMonitor
- Ping fileMonitor dans le conteneur file-generator

On peut faire la commande docker inspect file-generator et fileMonitor pour les adresses IP de chaque conteneur. Par exemple :

```
"MacAddress": "",
"Networks": {
  "monitoring_network": {
    "IPAMConfig": null,
    "Links": null,
    "Aliases": null,
    "MacAddress": "02:42:ac:12:00:03",
    "DriverOpts": null,
    "NetworkID": "0897ebb8ae94e5ac0688c6d7eb8b2f048116bf8ccc23407e87ec93bfaa1ff5c5",
    "EndpointID": "f9cecc5fe534c2e59afc87f9249f6f8d65d0a9bc5911f5c35012a47a7c14d502",
    "Gateway": "172.18.0.1",
    "IPAddress": "172.18.0.3",
    "IPPrefixLen": 16,
    "IPv6Gateway": "",
    "GlobalIPv6Address": "",
    "GlobalIPv6PrefixLen": 0,
    "DNSNames": [
      "file-generator",
      "d6a6e837436f"
    ]
  }
}
```

6. a)

- Ls -l > /app/data/file_inventory.txt
- */5 * * * * /bin/bash /app/data/listes_fichier.sh

b)

```
root@0df57a07754a: /
GNU nano 7.2 /app/data/verif_file.sh
if [ -f /app/data/file_inventory.txt ]; then
cp /app/data/file_inventory.txt /app/data/file_inventory_$(date +%Y_%m_%d_%H:%M).txt
echo "Copie du fichier creee : file_inventory_$(date +%Y_%m_%d_%H:%M)" >> /app/data/file_monitoring.log
fi
```

```
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirect
#
*/2 * * * * /bin/bash /app/data/verif_file.sh
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -czf /var/backups/home.tar.gz /home/
```

7.

```
file_inventory_2024_11_06_00:20.txt file_inventory_2024_11_06_00:47.txt file_inventory_2024_11_11_17:47.txt
file_inventory_2024_11_06_00:30.txt file_inventory_2024_11_11_17:47.txt file_inventory_2024_11_11_17:54.txt
root@0df57a07754a: /# ls app/data file_inventory_2024_11_06_00:34.txt file_inventory_2024_11_11_17:56.txt
file_inventory.txt file_inventory_2024_11_06_00:36.txt file_inventory_2024_11_11_17:58.txt
file_inventory.txt_2024_11_06_00-23.txt file_inventory_2024_11_06_00:38.txt file_inventory_2024_11_11_18:00.txt
file_inventory_2024_11_06_00:24.txt file_inventory_2024_11_06_00:40.txt file_inventory_2024_11_11_18:02.txt
file_inventory_2024_11_06_00:25.txt file_inventory_2024_11_06_00:42.txt file_monitoring.log
file_inventory_2024_11_06_00:26.txt file_inventory_2024_11_06_00:44.txt liste_fichier.sh
file_inventory_2024_11_06_00:27.txt file_inventory_2024_11_11_17:47.txt verif_file.sh
file_inventory_2024_11_06_00:28.txt file_inventory_2024_11_11_17:50.txt
file_inventory_2024_11_06_00:30.txt file_inventory_2024_11_11_17:52.txt
file_inventory_2024_11_06_00:32.txt
root@0df57a07754a: /# cat /app/data/file_monitoring.log
Copie du fichier creee : file_inventory_2024_11_11_17:47
Copie du fichier creee : file_inventory_2024_11_11_17:50
Copie du fichier creee : file_inventory_2024_11_11_17:52
Copie du fichier creee : file_inventory_2024_11_11_17:54
Copie du fichier creee : file_inventory_2024_11_11_17:56
Copie du fichier creee : file_inventory_2024_11_11_17:58
Copie du fichier creee : file_inventory_2024_11_11_18:00
Copie du fichier creee : file_inventory_2024_11_11_18:02
root@0df57a07754a: /# |
```